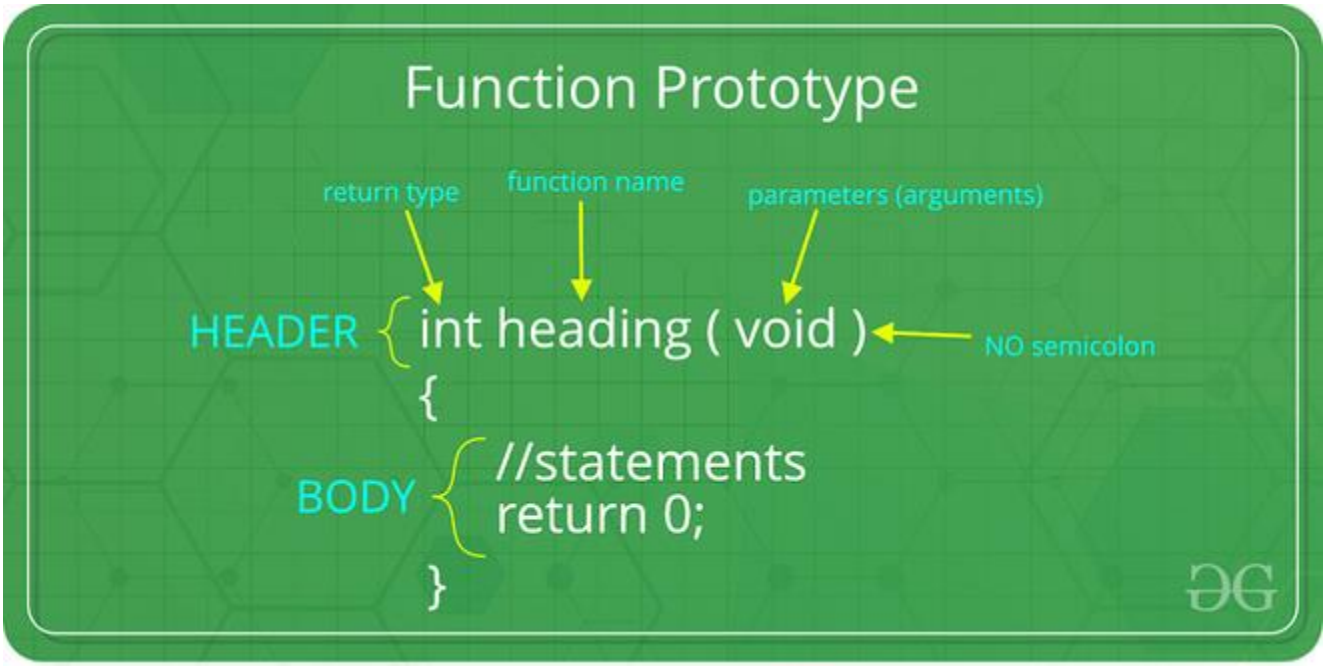
Functions in C++

A function is a set of statements that takes input, does some specific computation, and produces output. The idea is to put some commonly or repeatedly done tasks together to make a function so that instead of writing the same code again and again for different inputs, we can call this function. In simple terms, a function is a block of code that runs only when it is called.

Syntax:



Types of Functions:

1. **User Defined Function:** User-defined functions are user/customer-defined blocks of code specially customized to reduce the complexity of big programs. They are also commonly known as “tailor-made functions” which are built only to satisfy the condition in which the user is facing issues meanwhile reducing the complexity of the whole program.

2. **Library Function:** Library functions are also called “built-in Functions”. These functions are part of a compiler package that is already defined and consists of a special function with special and different meanings. Built-in Function gives us an edge as we can directly use them without defining them whereas in the user-defined function we have to declare and define a function before using them. For Example: sqrt(), setw(), strcat(), etc.

Parameter Passing to Functions:

The parameters passed to the function are called **actual parameters**. For example, in the program below, 5 and 10 are actual parameters.

The parameters received by the function are called **formal parameters**. For example, in the above program x and y are formal parameters.

```
class Multiplication {
   int multiply( int x, int y ) { return x * y; }
public
   static void main()
   {
      Multiplication M = new Multiplication();
      int gfg = 5, gfg2 = 10;
      int gfg3 = multiply( gfg, gfg2 );
      cout << "Result is " << gfg3;
   }
}
```

Formal Parameter

Actual Parameter

There are two most popular ways to pass parameters:

- **Pass by Value:** In this parameter passing method, values of actual parameters are copied to the function's formal parameters. The actual and formal parameters are stored in different memory locations, so any changes made in the functions are not reflected in the actual parameters of the caller.

- **Pass by Reference:** Both actual and formal parameters refer to the same locations, so any changes made inside the function are reflected in the actual parameters of the caller.

Difference between call by value and call by reference in C++

Call by value	Call by reference
A copy of the value is passed to the function	An address of value is passed to the function
Changes made inside the function are not reflected on other functions	Changes made inside the function are reflected outside the function as well
Actual and formal arguments will be created at different memory location	Actual and formal arguments will be created at same memory location.

Example 1: Display a Text

```
#include <iostream>
using namespace std;

// declaring a function
void greet() {
    cout << "Hello there!";
}

int main() {

    // calling the function
    greet();

    return 0;
}
```

Example 2: Function with Parameters

```
// program to print a text

#include <iostream>
using namespace std;

// display a number
void displayNum(int n1, float n2) {
    cout << "The int number is " << n1;
    cout << "The double number is " << n2;
}

int main() {

     int num1 = 5;
     double num2 = 5.5;

    // calling the function
    displayNum(num1, num2);

    return 0;
}
```

Example 3: Add Two Numbers

```
// program to add two numbers using a function

#include <iostream>

using namespace std;

// declaring a function
int add(int a, int b) {
    return (a + b);
}

int main() {

    int sum;

    // calling the function and storing
    // the returned value in sum
    sum = add(100, 78);

    cout << "100 + 78 = " << sum << endl;

    return 0;
}
```

Function Prototype

In C++, the code of function declaration should be before the function call. However, if we want to define a function after the function call, we need to use the function prototype. For example,

```
#include <iostream>

using namespace std;

// function prototype
int add(int, int);

int main() {
    int sum;

    // calling the function and storing
    // the returned value in sum
    sum = add(100, 78);

    cout << "100 + 78 = " << sum << endl;

    return 0;
}

// function definition
int add(int a, int b) {
    return (a + b);
}
```

Benefits of Using User-Defined Functions

- Functions make the code reusable. We can declare them once and use them multiple times.
- Functions make the program easier as each small task is divided into a function.
- Functions increase readability.

C++ User-defined Function Types

For better understanding of arguments and return values in functions, user-defined functions can be categorized into various types:

- Function with no argument and no return value
- Function with no argument but return value
- Function with argument but no return value
- Function with argument and return value

Consider a situation in which you have to check prime number. Let's do that by making user-defined functions in 4 different ways as mentioned above.

- **No Arguments Passed and No Return Value**

```cpp
#include <iostream>
using namespace std;
void check_prime();//declare a function with no arguments and no return value
int main() {
    // no argument is passed to prime()
    check_prime();
    return 0;
}
void check_prime() {
    int num;
    cout << "Enter a positive integer to check: ";
    cin >> num;
    bool is_prime = true;;
    for(int i = 2; i <= num/2; ++i) {
        if(num % i == 0) {
            is_prime = false;;
            break;
        }
    }
    if (is_prime) {
        cout << num << " is a prime number.";
    }
    else {
        cout << num << " is not a prime number.";
    }
}
```

- **No Arguments Passed but a Return Value**

```
#include <iostream>
using namespace std;
bool check_prime(); //declare a function with no arguments and return type int
int main() {
    // call prime and assign return value to is_prime
    bool is_prime = check_prime();
    if (is_prime) {
        cout << "Your number is a prime number.";
    }
    else {
        cout << "Your number is not a prime number.";
    }
    return 0;
}
bool check_prime() {
    int num;
    cout << "Enter a positive integer to check: ";
    cin >> num;
    bool is_prime = true;
    for (int i = 2; i <= num / 2; ++i) {
        if (num % i == 0) {
            is_prime = false;
            break;
        }
    }
    return is_prime;
}
```

H.W/Lab Task:

- Function with argument but no return value
- Function with argument and return value.

What is Function Overloading in C++?

Two or more functions can have the same name but different parameters; such functions are called function overloading in c++. The function overloading in the c++ feature is used to improve the readability of the code.

Rules of Function Overloading in C++

1. These functions have different parameter type
   sum(int a, int b)
   sum(double a, double b)
2. These functions have a different number of parameters
   sum(int a, int b)
   sum(int a, int b, int c)
3. These functions have a different sequence of parameters
   sum(int a, double b)
   sum(double a, int b)

Why is function overloading in C++ used?

Function overloading is similar to polymorphism that helps us to get different behavior, with the same name of the function. Function overloading in c++ is used for code reusability and to save memory.

Types of function overloading in c++

There are two types of function overloading in c++-

- **Compile time overloading–** In compile time overloading, functions are overloaded using different signatures. The signature of a function includes its return type, number of parameters and types of parameters.
- **Runtime overloading -**In runtime overloading, functions are overloaded using a different number of parameters.

Causes of function overloading in c++

- Type Conversion.
- Function with default arguments.
- Function with a pass-by reference.

- **Let's see a simple example with type conversion:**

```
#include<iostream>
using namespace std;
void fun(int);
void fun(double);
void fun(int i)
{
    std::cout << "Value of i is : " <<i<< std::endl;
}
void fun(double j)
{
    std::cout << "Value of j is : " <<j<< std::endl;
}
int main()
{
    fun(12);
    fun(1.2);
    return 0;
}
```

- **Let's see a simple example of function with default arguments:**

```
#include<iostream>
using namespace std;
// Function declaration for single-parameter fun
void funSingle(int);
// Function declaration for two-parameter fun (with default argument)
void funDouble(int a, int b = 9);
// Definition for funSingle (one int parameter)
void funSingle(int i)
{
    std::cout << "Value of i is : " << i << std::endl;
}
// Definition for funDouble (two int parameters)
void funDouble(int a, int b)
{
    std::cout << "Value of a is : " << a << std::endl;
    std::cout << "Value of b is : " << b << std::endl;
}
int main()
{
    funSingle(12);    // Calls funSingle(int)
    // To call funDouble(int, int) explicitly:
    funDouble(12, 9); // Calls funDouble(int, int) with b = 9
    return 0;
}
```

Lab task: Function with a pass-by reference

Advantages of Function Overloading in c++

- The program is very simple and also easier to understand.
- It saves memory space, maintains consistency, makes clear interface for methods regardless of the parameter's type and readability of the program.
- Using the function overloading concept, we can develop more than one function with the same name, but the arguments passed should be of different types.
- Function overloading makes the program faster.
- Function overloading is used for code reusability and to save memory.

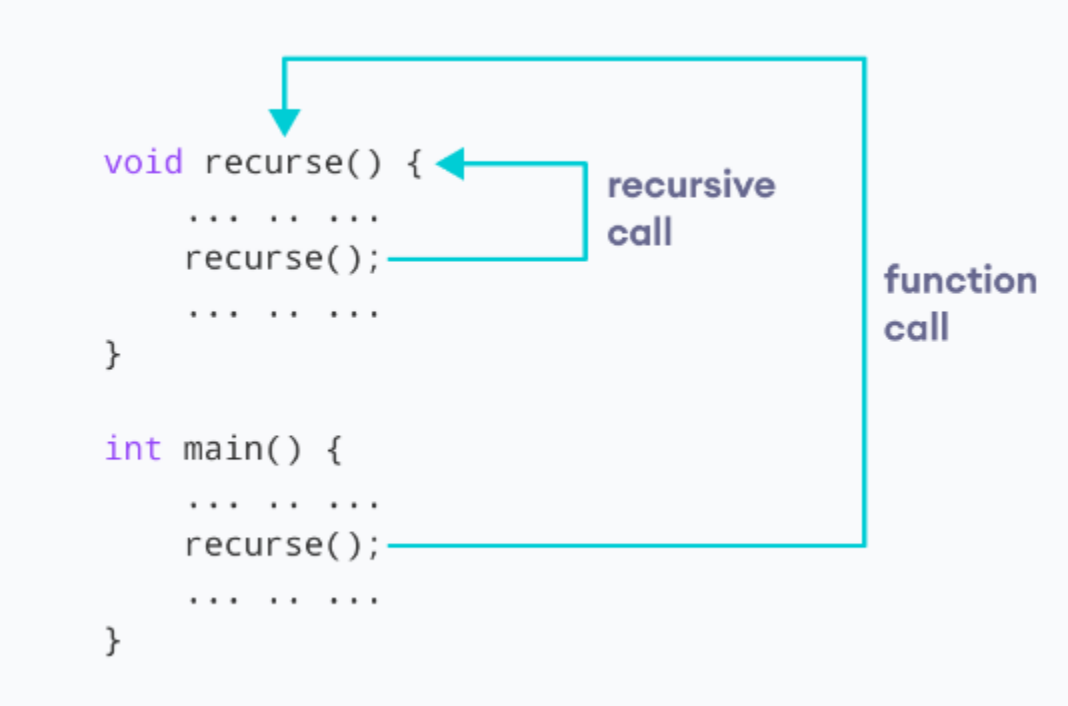
Disadvantages of Function Overloading in c++

- Function declarations that differ by their return type cannot be overloaded with the function overloading process.
- If any static member function is declared, then the same parameters or the same name types cannot be overloaded.

C++ Recursion

A function that calls itself is known as a recursive function. And this technique is known as recursion.

The figure below shows how recursion works by calling itself over and over again. The recursion continues until some condition is met.



Advantages of C++ Recursion

- It makes our code shorter and cleaner.
- Recursion is required in problems concerning data structures and advanced algorithms, such as Graph and Tree Traversal.

Disadvantages of C++ Recursion

- It takes a lot of stack space compared to an iterative program.
- It uses more processor time.
- It can be more difficult to debug compared to an equivalent iterative program.

Factorial of a Number Using Recursion

```
// Factorial of n = 1*2*3*...*n

#include <iostream>
using namespace std;

int factorial(int);

int main() {
    int n, result;

    cout << "Enter a non-negative number: ";
    cin >> n;

    result = factorial(n);
    cout << "Factorial of " << n << " = " << result;
    return 0;
}

int factorial(int n) {
    if (n > 1) {
        return n * factorial(n - 1);
    } else {
        return 1;
    }
}
```

```
return 5 * factorial(4) = 120
└── return 4 * factorial(3) = 24
    └── return 3 * factorial(2) = 6
        └── return 2 * factorial(1) = 2
            └── return 1 * factorial(0) = 1
```